CS4084
Quantum Computing
Week 14 – Lecture 27-28

Dr. Maqsood M. Khan

Department of Computer Science
FAST-NUCES Peshawar

Outline

This Week: The Path to Quantum Factoring I

We are now moving into the most mathematically elegant part of the course, where we use the **Spectral Theorem** to solve the **Order-Finding Problem**.

- **The Mathematical Core**
  - **Spectral Theorem:** Understanding why every unitary operator can be understood through its **Eigenvectors and Eigenvalues**.
- **The Phase Estimation Algorithm**
  - **Warm-up:** Revisiting **Phase Kickback** to extract information from an Oracle.
  - **Iterating Unitaries:** Using multiple control qubits to build precision.
  - **The Procedure:** A complete quantum circuit that estimates the phase of a unitary's eigenvalue.
- **The Quantum Fourier Transform (QFT)**
  - **Definition & Circuits:** The quantum version of the Discrete Fourier Transform.
  - **The Engine:** How the QFT acts as the "final lens" in Phase Estimation.

This Week: The Path to Quantum Factoring II

- **The Order-Finding & Factoring Problem**
  - **Problem Statement:** Finding the smallest integer r such that $a^r \equiv 1 \pmod{N}$.
  - **The Connection:** Using Phase Estimation on a "Random Eigenvector" to find the order r.
  - **Shor's Strategy:** How Factoring is mathematically reduced to Order-Finding.

Connection to Foundations

Remember the **Tensor Products** and **CNOTs** we studied? This week, we will see how n control qubits and a QFT circuit create a massive interference pattern that reveals the "hidden period" of a function.

Spectral Theorem

The *spectral theorem* is an important fact in linear algebra. Here is a statement of a special case of this theorem, for *unitary matrices.*

Spectral theorem for unitary matrices

Suppose U is an $N \times N$ unitary matrix.

There exists an orthonormal basis $\{|\psi_1\rangle, \ldots, |\psi_N\rangle\}$ of vectors along with complex numbers

$$\lambda_1 = e^{2\pi i \theta_1}, \ldots, \lambda_N = e^{2\pi i \theta_N}$$

such that

$$U = \sum_{k=1}^{N} \lambda_k |\psi_k\rangle\langle\psi_k|$$

Spectral Theorem

Each vector $|\psi_k\rangle$ is an *eigenvector* of U having *eigenvalue* λ_k:

$$U|\psi_k\rangle = \lambda_k|\psi_k\rangle = e^{2\pi i\theta_k}|\psi_k\rangle$$

Phase Estimation Problem

In the phase estimation problem, we're given two things:

1. A description of a *unitary quantum circuit* on n qubits.
2. An n-qubit *quantum state* $|\psi\rangle$.

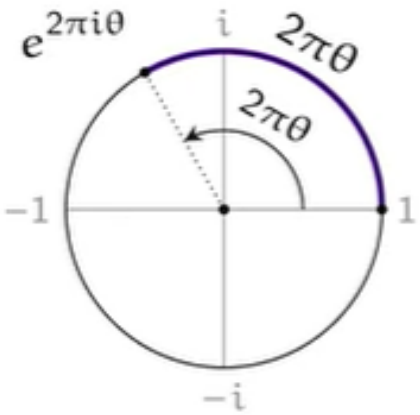
We're *promised* that $|\psi\rangle$ is an eigenvector of the unitary operation U described by the circuit, and our goal is to approximate the corresponding eigenvalue.

Phase estimation problem

Input:	A unitary quantum circuit for an n-qubit operation U and an n qubit quantum state $\|\psi\rangle$
Promise:	$\|\psi\rangle$ is an eigenvector of U
Output:	An approximation to the number $\theta \in [0, 1)$ satisfying

$$U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$$

Phase Estimation Problem



We can approximate θ by a fraction

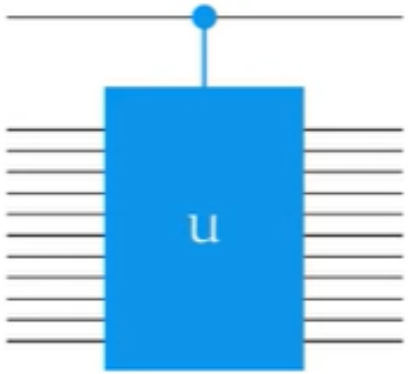
$$\theta \approx \frac{y}{2^m}$$

for $y \in \{0, 1, \ldots, 2^m - 1\}$.

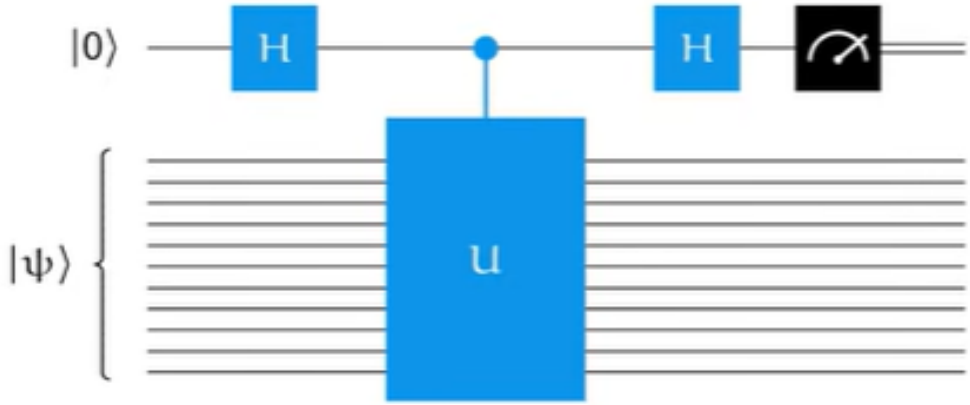
This approximation is taken "modulo 1."

Warm up: Using phase kickback!

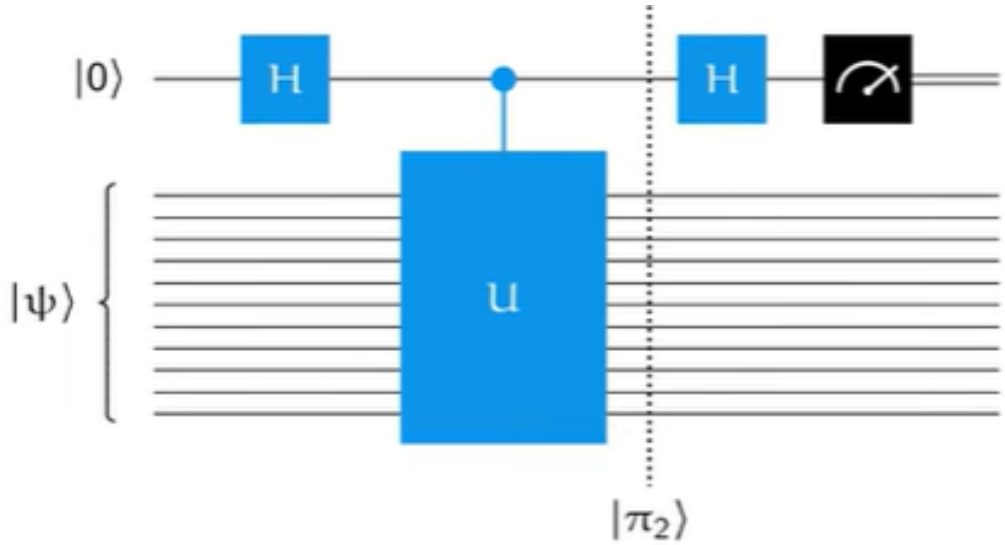
Given a circuit for U, we can create a circuit for a controlled-U operation:



Let's consider this circuit:



Warm up: Using phase kickback!

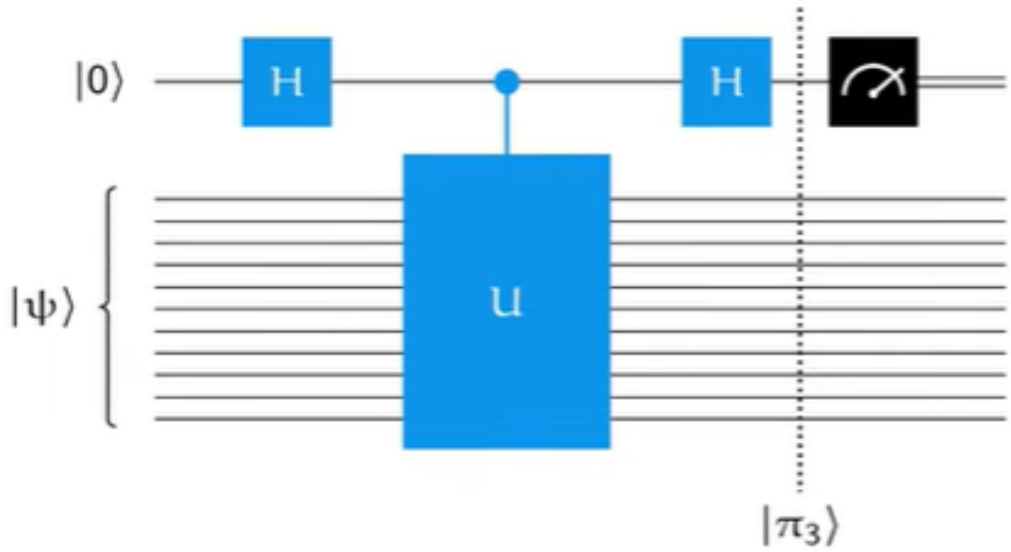


$$|\pi_0\rangle = |\psi\rangle|0\rangle$$

$$|\pi_1\rangle = \frac{1}{\sqrt{2}}|\psi\rangle|0\rangle + \frac{1}{\sqrt{2}}|\psi\rangle|1\rangle$$

$$|\pi_2\rangle = \frac{1}{\sqrt{2}}|\psi\rangle|0\rangle + \frac{1}{\sqrt{2}}(U|\psi\rangle)|1\rangle = |\psi\rangle \otimes \left(\frac{1}{\sqrt{2}}|0\rangle + \frac{e^{2\pi i\theta}}{\sqrt{2}}|1\rangle\right)$$

Warm up: Using phase kickback!



$$|\pi_2\rangle = |\psi\rangle \otimes \left(\frac{1}{\sqrt{2}}|0\rangle + \frac{e^{2\pi i\theta}}{\sqrt{2}}|1\rangle \right)$$

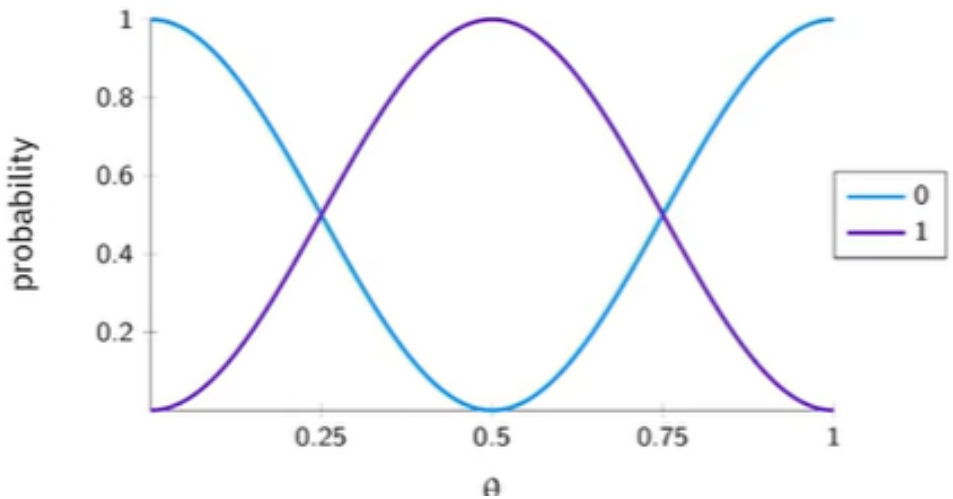
$$|\pi_3\rangle = |\psi\rangle \otimes \left(\frac{1+e^{2\pi i\theta}}{2}|0\rangle + \frac{1-e^{2\pi i\theta}}{2}|1\rangle \right)$$

Probabilities plots!

$$|\psi\rangle \otimes \left(\frac{1 + e^{2\pi i\theta}}{2} |0\rangle + \frac{1 - e^{2\pi i\theta}}{2} |1\rangle \right)$$

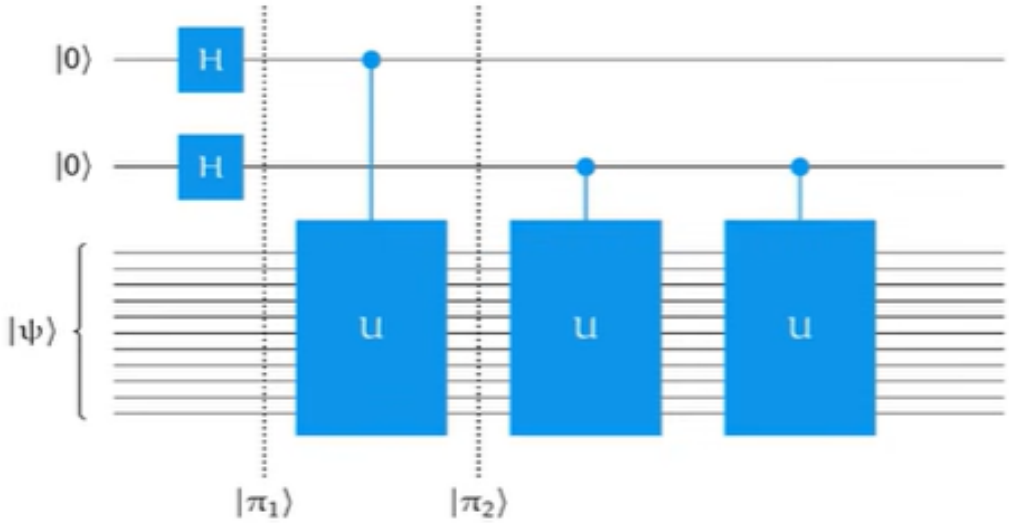
Measuring the top qubit yields the outcomes 0 and 1 with these probabilities:

$$p_0 = \left| \frac{1 + e^{2\pi i\theta}}{2} \right|^2 = \cos^2(\pi\theta) \qquad p_1 = \left| \frac{1 - e^{2\pi i\theta}}{2} \right|^2 = \sin^2(\pi\theta)$$



Two control qubit

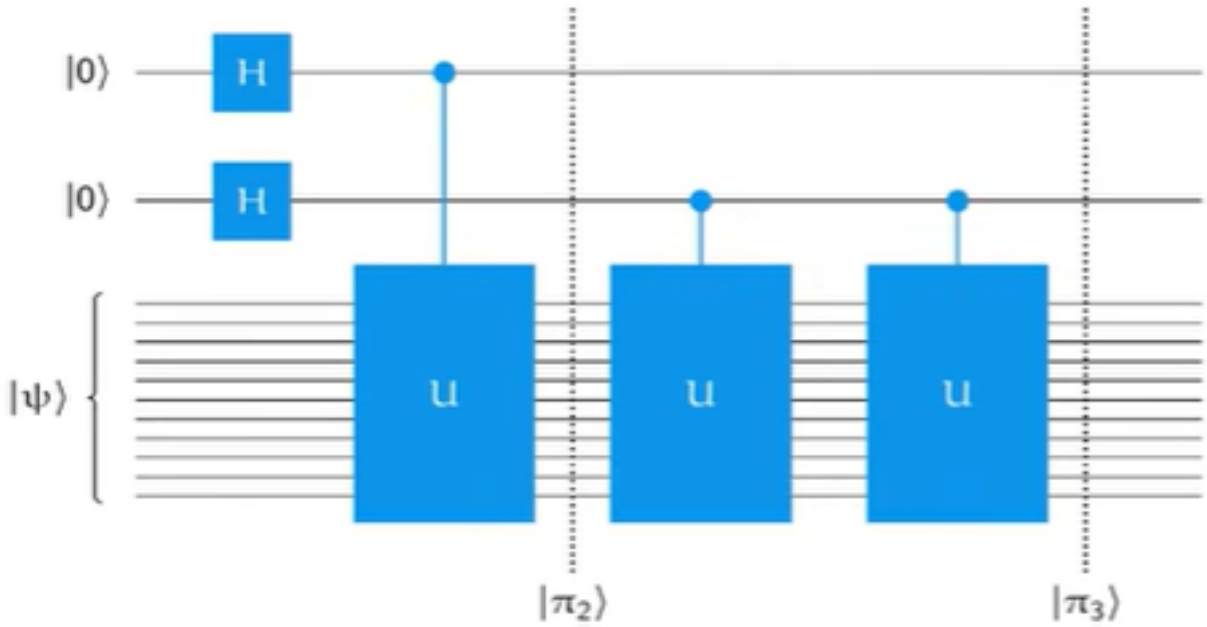
Let's use two control qubits to perform the controlled-U operations — and then we'll see how best to proceed.



$$|\pi_1\rangle = |\psi\rangle \otimes \frac{1}{2} \sum_{a_0=0}^{1} \sum_{a_1=0}^{1} |a_1 a_0\rangle$$

$$|\pi_2\rangle = |\psi\rangle \otimes \frac{1}{2} \sum_{a_0=0}^{1} \sum_{a_1=0}^{1} e^{2\pi i a_0 \theta} |a_1 a_0\rangle$$

Two control qubit



$$|\pi_2\rangle = |\psi\rangle \otimes \frac{1}{2} \sum_{a_0=0}^{1} \sum_{a_1=0}^{1} e^{2\pi i a_0 \theta} |a_1 a_0\rangle$$

$$|\pi_3\rangle = |\psi\rangle \otimes \frac{1}{2} \sum_{a_0=0}^{1} \sum_{a_1=0}^{1} e^{2\pi i (2a_1 + a_0) \theta} |a_1 a_0\rangle$$

Two control qubit

$$|\pi_3\rangle = |\psi\rangle \otimes \frac{1}{2} \sum_{a_0=0}^{1} \sum_{a_1=0}^{1} e^{2\pi i(2a_1+a_0)\theta} |a_1 a_0\rangle$$

$$= |\psi\rangle \otimes \frac{1}{2} \sum_{x=0}^{3} e^{2\pi i x\theta} |x\rangle$$

Two control qubit

$$\frac{1}{2}\sum_{x=0}^{3} e^{2\pi i x\theta}|x\rangle$$

What can we learn about θ from this state? Suppose we're promised that $\theta = \frac{y}{4}$ for $y \in \{0, 1, 2, 3\}$. Can we figure out which one it is?

Define a two-qubit state for each possibility:

$$|\phi_y\rangle = \frac{1}{2}\sum_{x=0}^{3} e^{2\pi i \frac{xy}{4}}|x\rangle$$

$$|\phi_0\rangle = \frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle + \frac{1}{2}|2\rangle + \frac{1}{2}|3\rangle$$

$$|\phi_1\rangle = \frac{1}{2}|0\rangle + \frac{i}{2}|1\rangle - \frac{1}{2}|2\rangle - \frac{i}{2}|3\rangle$$

$$|\phi_2\rangle = \frac{1}{2}|0\rangle - \frac{1}{2}|1\rangle + \frac{1}{2}|2\rangle - \frac{1}{2}|3\rangle$$

$$|\phi_3\rangle = \frac{1}{2}|0\rangle - \frac{i}{2}|1\rangle - \frac{1}{2}|2\rangle + \frac{i}{2}|3\rangle$$

These vectors are *orthonormal* — so they can be discriminated perfectly by a projective measurement.

Two control qubit

$$|\phi_y\rangle = \frac{1}{2}\sum_{x=0}^{3} e^{2\pi i \frac{xy}{4}}|x\rangle$$

$$|\phi_0\rangle = \frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle + \frac{1}{2}|2\rangle + \frac{1}{2}|3\rangle$$
$$|\phi_1\rangle = \frac{1}{2}|0\rangle + \frac{i}{2}|1\rangle - \frac{1}{2}|2\rangle - \frac{i}{2}|3\rangle$$
$$|\phi_2\rangle = \frac{1}{2}|0\rangle - \frac{1}{2}|1\rangle + \frac{1}{2}|2\rangle - \frac{1}{2}|3\rangle$$
$$|\phi_3\rangle = \frac{1}{2}|0\rangle - \frac{i}{2}|1\rangle - \frac{1}{2}|2\rangle + \frac{i}{2}|3\rangle$$

$$V = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}$$

The unitary matrix V whose *columns* are $|\phi_0\rangle$, $|\phi_1\rangle$, $|\phi_2\rangle$, $|\phi_3\rangle$ has this action:

$$V|y\rangle = |\phi_y\rangle \quad \text{(for every } y \in \{0,1,2,3\}\text{)}$$

We can identify y by performing the inverse of V then a standard basis measurement.

$$V^\dagger|\phi_y\rangle = |y\rangle \quad \text{(for every } y \in \{0,1,2,3\}\text{)}$$

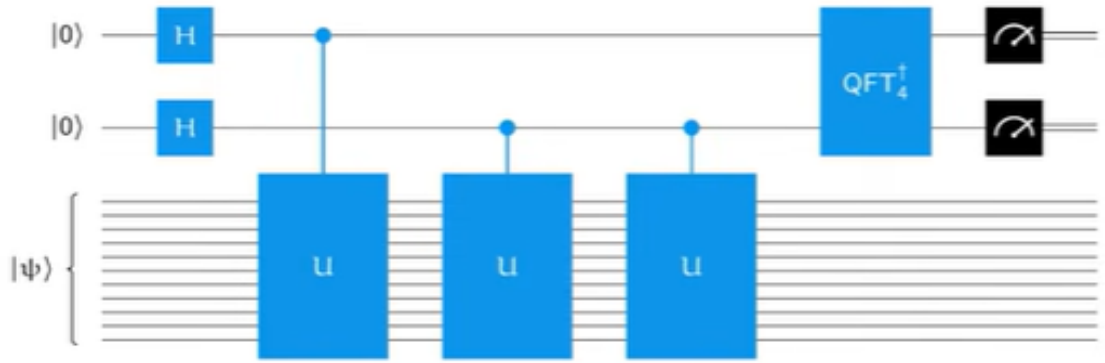
Quantum Fourier Transform obtained from v

$$\mathrm{QFT}_4 = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}$$

This matrix is associated with the *discrete Fourier transform* (for 4 dimensions).

When we think about this matrix as a unitary operation, we call it the *quantum Fourier transform.*

The complete circuit for learning $y \in \{0, 1, 2, 3\}$ when $\theta = y/4$:



The Quantum Fourier Transform (QFT): What and Why?

The QFT is the quantum implementation of the Discrete Fourier Transform (DFT). It is perhaps the most important subroutine in quantum computing.

- **What is it?**
  - It transforms a quantum state from the **Computational Basis** (time/position) to the **Fourier Basis** (frequency/phase).
  - Mathematically: $|j\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle$.
- **Why do we use it?**
  - **Extracting Periodicity:** It turns hidden periodic patterns (like the order r in Shor's algorithm) into measurable peaks.
  - **Phase Extraction:** In Phase Estimation, information is stored in the "phases" of qubits. The QFT "unpacks" these phases into readable bitstrings.
- **Quantum Advantage:**
  - **Classical (FFT):** Takes $O(N \log N)$ steps, where $N = 2^n$.
  - **Quantum (QFT):** Takes only $O(n^2)$ steps. This is an **exponential speedup** (n vs 2^n).

Visual Intuition

If a standard gate is like a "note" on a piano, the QFT is the "ear" that listens to the entire chord and tells you which frequencies are being played.

Quantum Fourier Transform obtained from v

The quantum Fourier transform is defined for each positive integer N as follows.

$$\mathrm{QFT}_N = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} e^{2\pi i \frac{xy}{N}} |x\rangle\langle y|$$

$$\mathrm{QFT}_N|y\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} e^{2\pi i \frac{xy}{N}} |x\rangle$$

Example

$$\mathrm{QFT}_8 = \frac{1}{2\sqrt{2}} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \frac{1+i}{\sqrt{2}} & i & \frac{-1+i}{\sqrt{2}} & -1 & \frac{-1-i}{\sqrt{2}} & -i & \frac{1-i}{\sqrt{2}} \\ 1 & i & -1 & -i & 1 & i & -1 & -i \\ 1 & \frac{-1+i}{\sqrt{2}} & -i & \frac{1+i}{\sqrt{2}} & -1 & \frac{1-i}{\sqrt{2}} & i & \frac{-1-i}{\sqrt{2}} \\ 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & \frac{-1-i}{\sqrt{2}} & i & \frac{1-i}{\sqrt{2}} & -1 & \frac{1+i}{\sqrt{2}} & -i & \frac{-1+i}{\sqrt{2}} \\ 1 & -i & -1 & i & 1 & -i & -1 & i \\ 1 & \frac{1-i}{\sqrt{2}} & -i & \frac{-1-i}{\sqrt{2}} & -1 & \frac{-1+i}{\sqrt{2}} & i & \frac{1+i}{\sqrt{2}} \end{pmatrix}$$

Matrix Examples: QFT_1 and QFT_2

The QFT can be represented as an $N \times N$ matrix where the entry at row j and column k is $\omega^{jk}/\sqrt{N}$, with $\omega = e^{2\pi i/N}$.

- **1-Dimensional QFT ($N = 1$):** For a single dimension (not a qubit), the transform is trivial:

$$QFT_1 = (1) \tag{1}$$

- **2-Dimensional QFT ($N = 2$):** This corresponds to a single qubit ($n = 1$) and is identical to the **Hadamard Matrix** (H):

$$QFT_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = H \tag{2}$$

Observation

Notice that for $N = 2$, the phase factor $e^{2\pi i(1)(1)/2} = e^{i\pi} = -1$.

Matrix Examples: QFT_3 and QFT_4

As the dimension increases, the complex roots of unity begin to fill the matrix.

- **3-Dimensional QFT ($N = 3$):** Using $\omega = e^{2\pi i/3} = \frac{-1+i\sqrt{3}}{2}$:

$$QFT_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 1 & 1 \\ 1 & \frac{-1+i\sqrt{3}}{2} & \frac{-1-i\sqrt{3}}{2} \\ 1 & \frac{-1-i\sqrt{3}}{2} & \frac{-1+i\sqrt{3}}{2} \end{pmatrix} \tag{3}$$

- **4-Dimensional QFT ($N = 4$):** This corresponds to two qubits ($n = 2$), where $\omega = e^{2\pi i/4} = i$:

$$QFT_4 = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix} \tag{4}$$

Matrix Symmetry

Note that these matrices are symmetric and unitary ($U^\dagger U = I$), which ensures that the total probability of the quantum state is conserved.

Quantum Fourier Transform

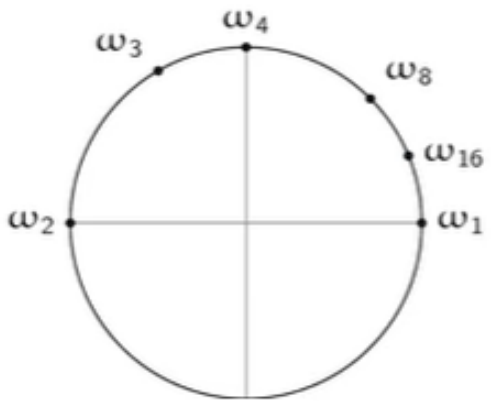
The quantum Fourier transform is defined for each positive integer N as follows.

$$\mathrm{QFT}_N = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} e^{2\pi i \frac{xy}{N}} |x\rangle\langle y|$$

$$\mathrm{QFT}_N |y\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} e^{2\pi i \frac{xy}{N}} |x\rangle$$

Useful shorthand notation:

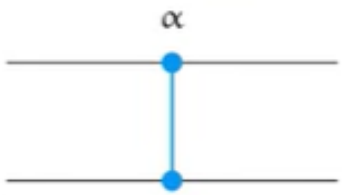
$$\omega_N = e^{\frac{2\pi i}{N}} = \cos\left(\frac{2\pi}{N}\right) + i \sin\left(\frac{2\pi}{N}\right)$$



Circuits for QFT

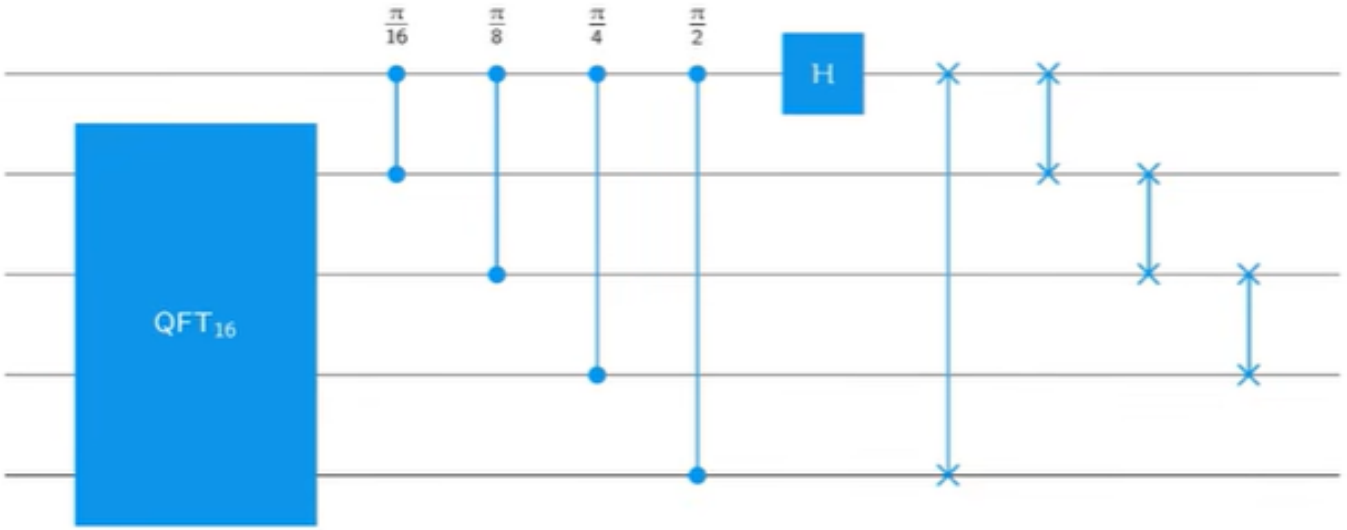
We can implement QFT_N efficiently with a quantum circuit when N is a power of 2.

The implementation makes use of *controlled-phase* gates:



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\alpha} \end{pmatrix}$$

The implementation is *recursive* in nature. As an example, here is the circuit for QFT_{32}:



Generating QFT_8: The 3-Qubit Algorithm

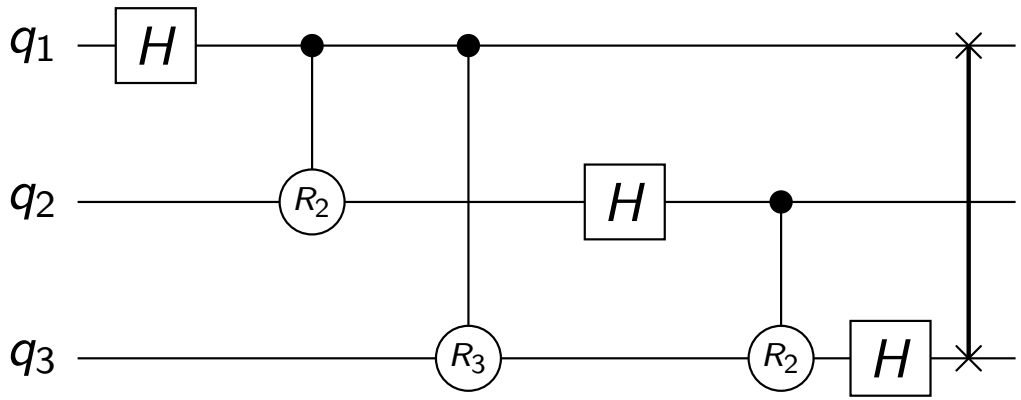
Building the circuit for $N = 8$ ($n = 3$ qubits) follows a recursive pattern of rotations and interference.

Step-by-Step Construction:

- **Stage 1 (Qubit 1):** Apply a Hadamard (H) gate to q_1, followed by controlled-phase rotations from the remaining qubits:
  - CR_2 (controlled by q_2): Phase shift of $\pi/2$.
  - CR_3 (controlled by q_3): Phase shift of $\pi/4$.
- **Stage 2 (Qubit 2):** Apply H to q_2, followed by a controlled-phase rotation from the remaining qubit:
  - CR_2 (controlled by q_3): Phase shift of $\pi/2$.
- **Stage 3 (Qubit 3):** Apply the final H gate to q_3.
- **Final Stage:** Perform a **SWAP** between q_1 and q_3 to correct the bit-reversal order inherent in the QFT.

Circuit Diagram for QFT_8

The recursive nature of the implementation uses controlled-phase gates to build the frequency spectrum.



The Rotation Gate Definition

The R_k gate is defined as: $R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$.

Number of gates required for the QFT implementation!

Cost analysis

Let s_m denote the number of gates we need for m qubits.

- For $m = 1$, a single Hadamard gate is required.
- For $m \geq 2$, these are the gates required:

  s_{m-1} gates for the QFT on $m - 1$ qubits
  $m - 1$ controlled phase gates
  $m - 1$ swap gates
  1 Hadamard gate

$$s_m = \begin{cases} 1 & m = 1 \\ s_{m-1} + 2m - 1 & m \geq 2 \end{cases}$$

This is a *recurrence relation* with a closed-form solution:

$$s_m = \sum_{k=1}^{m} (2k - 1) = m^2$$

Why Shor's? The Exponential Gap

The primary motivation for Shor's Algorithm is the "Factoring Problem," which protects modern digital security (RSA).

- **The Classical Barrier:**
  - The best classical algorithm (General Number Field Sieve) is **sub-exponential**.
  - For a 2048-bit number, a classical supercomputer would take **trillions of years** to factor it.
- **The Quantum Leap:**
  - Shor's Algorithm is **Polynomial** ($O(n^3)$).
  - That same 2048-bit number could theoretically be factored in **hours** or days.

The "Magic" of the Algorithm

Shor's doesn't just "guess" faster. It uses **Quantum Parallelism** and the **Quantum Fourier Transform** to find a hidden mathematical period that classical computers must search for bit-by-bit.

Verification: How Factoring becomes Period-Finding

To factor $N = 15$, we pick a random $a = 7$ and find the period r of $7^x \pmod{15}$.

x	1	2	3	**4**	5	6
$7^x \pmod{15}$	7	4	13	**1**	7	4

The Verification Steps:

1. **Identify Period:** The sequence repeats every 4 steps $\rightarrow r = 4$.
2. **Half-Period Calculation:** $a^{r/2} = 7^{4/2} = 49$.
3. **Compute Factors:**
   - $\gcd(49 - 1, 15) = \gcd(48, 15) = \mathbf{3}$
   - $\gcd(49 + 1, 15) = \gcd(50, 15) = \mathbf{5}$

Conclusion

Because $3 \times 5 = 15$, the quantum-found period r successfully cracked the number.

Shor's Algorithm: A Hybrid Architecture

Shor's algorithm is the most famous example of **Quantum Speedup**. It reduces a difficult math problem (Factoring) into a physical problem (Period Finding).

- **Classical Part:**
  - Pick random $a < N$. If $\gcd(a, N) > 1$, we are done.
  - Otherwise, find the period r of $f(x) = a^x \pmod{N}$.
- **Quantum Part:** Use **Quantum Phase Estimation** to find r.
- **Post-Processing:** Use r to find factors via $\gcd(a^{r/2} \pm 1, N)$.

The Logic

If we can find the period r of the function, we can "break" the number N into its prime components.

How Shor's Uses Quantum Phase Estimation (QPE)

Shor's Algorithm is essentially QPE applied to a specific Unitary operator U.

- **The Operator (U):** We define U such that $U|y\rangle = |ay \pmod N\rangle$.
- **The Eigenvalues:** The eigenvalues of this operator are of the form $e^{2\pi i(s/r)}$, where r is the period we need.
- **The Goal of QPE:** Extract the phase $\theta = s/r$.

Circuit Logic:

1. Create a superposition of inputs in Register 1.
2. Apply U controlled by Register 1 (Modular Exponentiation).
3. This "kicks back" the phase into Register 1.

The Role of the QFT in Shor's Algorithm

The **Quantum Fourier Transform** is the final "lens" that makes the answer readable.

- **The Interference Pattern:** After modular exponentiation, Register 1 contains a state where the phases are rotating at a frequency related to $1/r$.
- **From Phase to State:** We cannot measure a phase directly. The **Inverse QFT** ($QFT^{\dagger}$) converts these rotating phases into a physical bitstring $|k\rangle$.
- **The Result:** Measuring the qubits gives us an integer $k \approx \frac{2^n \cdot s}{r}$. We then use classical continued fractions to solve for r.

Why it's Fast

Classical computers check values of x one by one to find the period. QFT finds the "frequency" of the entire superposition in $O(n^2)$ steps.

Step-by-Step Example: Factoring $N = 15$

Let's factor $N = 15$ with $a = 7$:

1. **The Sequence:** $7^1 = 7, 7^2 = 4, 7^3 = 13, 7^4 = 1 \pmod{15}$.
2. **The Period:** The values repeat every 4 steps, so $r = 4$.
3. **Quantum Measurement:** The QPE/QFT output will point toward phases $0, 0.25, 0.5, 0.75$ (multiples of $1/r$).
4. **Final Calculation:**
   - $7^{4/2} + 1 = 50 \rightarrow \gcd(50, 15) = \mathbf{5}$
   - $7^{4/2} - 1 = 48 \rightarrow \gcd(48, 15) = \mathbf{3}$

Success!

We successfully found the factors 3 and 5.

Lecture Summary: Key Concepts Covered I

- **The Query Model:** Measuring efficiency by the number of Oracle (black-box) queries rather than individual gate operations.
- **Phase Kickback:** The fundamental "trick" where the output of an oracle is kicked back into the phase of the input qubit, enabling interference.
- **Algorithm Evolution:**
  - **Deutsch-Jozsa:** Determining global function properties (constant vs. balanced) in one query.
  - **Bernstein-Vazirani:** Finding a hidden bit string s with a single query.
  - **Simon's Algorithm:** Finding a hidden period; the first proof of exponential speedup over classical bit-by-bit checking.

Lecture Summary: Key Concepts Covered II

- **The Spectral Theorem:** The mathematical bridge ensuring every Unitary operator U can be understood through its Eigenvectors and Eigenvalues ($U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$).
- **Quantum Fourier Transform (QFT):**
  - The "Frequency Scanner" of quantum computing.
  - Provides exponential speedup ($O(n^2)$ vs $O(2^n)$) for finding patterns in superposition.
  - **Matrix Form:** Generalized Hadamard matrices ($QFT_2 = H$) involving complex roots of unity.
- **Quantum Phase Estimation (QPE):** The high-level framework that uses QFT to "read" the hidden phase θ inside a Unitary gate.
- **Shor's Algorithm:**
  - **The Breakthrough:** Factoring large integers N in polynomial time, rendering RSA encryption vulnerable.
  - **The Mechanism:** Reduces factoring to a **Period-Finding** problem.
  - **Verification ($N = 15$):** Using $a = 7$, we find period $r = 4$, leading to factors $\gcd(7^{4/2} \pm 1, 15) \rightarrow \{3, 5\}$.

Lecture Summary: Key Concepts Covered III

- **Implementation Reality:** While theoretically powerful, these algorithms (especially modular exponentiation) are highly sensitive to **NISQ noise** (Depolarizing/Phase Damping).